

---

# HOW CAN EVERY ROLLOUT COUNT FOR SKILL SEEKERS?

## COMPASS FOR SKILL PROPOSERS

**Anonymous authors**

Paper under double-blind review

### ABSTRACT

Textual-program optimizers use execution traces to revise prompts, instructions, and agent skills. Under a fixed rollout budget, candidate generation and candidate verification draw from the same evidence supply. The number of independent proposal–verification attempts determines search bandwidth, and fresh support for each fixed candidate determines verification precision. Adaptive evidence reuse also makes provenance part of the optimization problem. This paper formulates *provenance-constrained evidence routing* with terminal simple regret as its objective. COMPASS allocates finite search bandwidth and verification support while preserving the generation–certification boundary across complete lineages. Recursive exclusion assigns each candidate a legal evidence set. Instance-relative credit compares legally observed candidates on the same instance and updates with the reward frontier. A tie-inclusive persistent queue and maximum-weight proposal allocation distribute search slots across distinct repair opportunities. Each fixed child then receives an independently sampled legal admission batch. Under the shared budget, more independent attempts reduce the search gap, and more fresh support increases the precision of candidate selection. Completed Qwen3-8B held-out owner evaluations span mathematical reasoning (e.g., AIME-2025), instruction following (IFBench), multi-hop question answering and fact verification (HotPotQA and HoVer), and privacy-conscious delegation (PUPA). Relative to published GEPA, the largest gain is 25.3 percentage points on AIME-2025, and the mean gain across all six shared benchmarks is 7.3 points. These results show that governing how evidence is created, reused, and routed can materially affect sample-efficient textual-program evolution.

## 1 INTRODUCTION

Feedback-driven optimizers revise instructions, prompts, and agent skills from execution traces. Reflection, textual gradients, and evolutionary prompt search use language feedback to produce large behavioral updates with fixed model weights (Shinn et al., 2023; Madaan et al., 2023; Yuksekgonul et al., 2025; Cheng et al., 2024; Agrawal et al., 2025). Their common optimization task is compact: given a seed textual program, an owner evaluator, and rollout budget  $B$ , the optimizer adaptively generates and evaluates candidates, then returns one terminal program. Every rollout serves one of two statistical roles. Proposal evidence diagnoses a failure and shapes a revision. Verification evidence evaluates a revision whose text is already fixed. Three instruction-following cases, for example, may expose a constraint-ordering error and inspire a new rule. Fresh owner cases then measure the fixed rule.

Independent proposal–verification attempts set search bandwidth, and fresh support per fixed candidate sets ranking precision. Their shared budget trades broader search against sharper admission and terminal selection. Lineage provenance governs the eligible evidence for verification: every batch that shapes a candidate becomes proposal history for its descendants. The optimizer routes this finite budget under a recursive generation–certification boundary.

This paper studies this task as *provenance-constrained evidence routing*. The routing policy minimizes expected terminal simple regret under a total rollout budget, finite search bandwidth, finite fresh verification support, and lineage-level provenance. A committed partial reward table supplies legal routing evidence. The single objective is to return the strongest terminal program supported by the budget and owner evaluation.

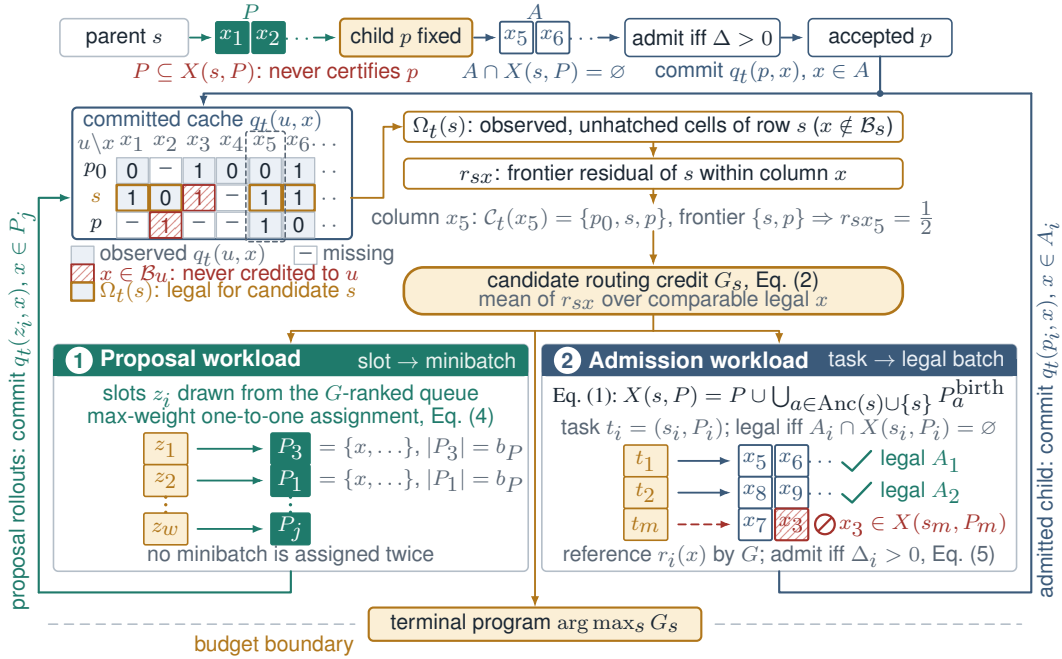


Figure 1: **Provenance-constrained evidence routing in COMPASS.** Top: one attempt proposes child  $p$  from parent  $s$  on minibatch  $P$  (green chips are instances  $x \in P$ ), certifies the fixed child on an admission batch  $A$  (blue chips,  $x \in A$ ) sampled outside the recursive exclusion set  $X(s, P)$  of Equation (1), and commits the accepted child's admission observations into the cache. Middle left: the committed cache is the partial table  $q_t(u, x)$  of programs (rows) by instances (columns); a hatched red cell is an observation stored for  $u$  but never credited to  $u$  because  $x$  lies in  $u$ 's lineage birth set  $\mathcal{B}_u$ , and the amber-framed cells of row  $s$  form its legal evidence set  $\Omega_t(s)$ . Middle right: each legal observation yields a within-instance frontier residual  $r_{sx}$  (worked for column  $x_5$ ), and their mean over comparable instances is the routing credit  $G_s$  of Equation (2). Bottom:  $G_s$  ranks the persistent slots  $z_i$  that the assignment in Equation (4) couples to distinct proposal minibatches  $P_j$ , and selects references  $r_i(x)$  for each task's legal admission batch  $A_i$ ; a batch that meets  $X(s_i, P_i)$  is refused. The labelled outer lanes return both kinds of observation to the cache. Red always marks one condition: an instance inside the birth provenance of the lineage in question. The terminal program is the maximal defined  $G_s$ .

Three challenges govern this objective. First, birth provenance must propagate through the full lineage so that each fixed child receives fresh certification evidence. Second, a partial and uneven reward table needs a candidate credit that remains comparable across instance difficulty and updates with new frontiers. Third, finite proposal capacity must reach distinct, high-value repair opportunities across concurrent search paths.

COMPASS (Credit-Aware Optimization with Mini-batch Proposal and Admission for Skill Search) implements this routing principle through three aligned mechanisms. Recursive exclusion propagates the generation-certification boundary and supplies each fixed child with an owner-sampled legal admission batch. Instance-relative credit compares legally observed candidates on the same instance and refreshes their ordering whenever the reward frontier changes. The tie-inclusive Top- $\rho$  envelope and persistent queue preserve candidate coverage, while a maximum-weight one-to-one allocation sends proposal slots to distinct repair opportunities. Figure 1 shows how these mechanisms route limited evidence from candidate generation to terminal selection.

Completed owner evaluations span model families and task types. On Qwen3-8B, task-specific COMPASS-family variants reach 57.33% on AIME-2025, 43.37% on IFBench, 66.67% on LiveBench-Math, and 51.33% on HoVer. The first three exceed published GEPA by 25.33, 4.76, and 14.72 percentage points; HoVer is 1.00 point lower. Each endpoint has an exact rollout, candidate,

token, and cost ledger. The frozen checkpoints retain lineage, queue, allocation, and cache-activation state for mechanism analysis.

This paper makes four contributions:

1. *Provenance-constrained evidence routing* casts textual-program evolution as terminal simple-regret minimization under explicit constraints on rollout budget, search bandwidth, verification support, and lineage provenance (Section 2).
2. COMPASS recursively separates generation from certification and turns committed legal evidence into a routing signal based on how each candidate compares with its peers on the same instance. The signal is recomputed as the partial observation state changes (Sections 3.1 and 3.2).
3. A tie-inclusive Top- $\rho$  envelope supplies persistent parent slots, and an exact maximum-weight integer assignment couples those slots to repairable proposal batches. Each fixed child receives its own provenance-legal owner batch and paired reference gate (Section 3.3).
4. Frozen COMPASS-family identities are evaluated under fixed published budgets across models and tasks, including mathematical reasoning, instruction following, and multi-hop fact verification (Section 5).

## 2 PROVENANCE-CONSTRAINED EVIDENCE ROUTING

### 2.1 BUDGETED TEXTUAL-PROGRAM OPTIMIZATION

Given a seed textual program  $p_0$ , an owner evaluator, and rollout budget  $B$ , a routing policy  $\pi$  adaptively generates and evaluates candidates and returns a terminal program  $p_{\hat{s}_\pi}$ . Let  $R(p)$  denote owner utility and let  $R^*$  denote the best reachable utility under the search model. The budgeted objective is

$$\pi^* \in \arg \min_{\pi \in \Pi(B, b_P, b_A)} \mathbb{E}[R^* - R(p_{\hat{s}_\pi})],$$

where  $b_P, b_A \in \mathbb{N}_+$ , with  $|P| = b_P$  proposal observations and  $|A| = b_A$  fresh verification observations per attempt. Policies in  $\Pi(B, b_P, b_A)$  divide their rollout expenditure as

$$C_{\text{proposal}}(\pi) + C_{\text{verify}}(\pi) \leq B.$$

Proposal expenditure determines the number  $M_B(\pi)$  of independently proposed-and-certified children, which is the search bandwidth. Verification expenditure assigns  $b_A$  fresh owner observations to each fixed child and controls the precision of admission and terminal selection. Larger  $M_B(\pi)$  lowers the search gap, and larger  $b_A$  lowers selection uncertainty.

At optimization step  $t$ , let  $\mathcal{P}_t = \{p_0, \dots, p_{m_t-1}\}$  contain the accepted textual programs. Every candidate after  $p_0$  records a parent, inducing a directed acyclic lineage. The *owner* is the benchmark-authoritative data, execution, metric, and aggregation contract. Executing program  $p_s$  on optimization instance  $x$  yields a bounded reward in  $[0, 1]$ , an output, and an execution trace. The committed cache is a partial table  $q_t(s, x)$  containing the best finite owner-accepted observation for each evaluated pair, with unevaluated pairs represented by missing entries. Let  $P_a^{\text{birth}}$  be the proposal batch that created candidate  $a$ , with  $P_0^{\text{birth}} = \emptyset$ , and let  $\text{Anc}(s)$  contain the transitive ancestors of  $s$ . The lineage birth set is

$$\mathcal{B}_s = \bigcup_{a \in \text{Anc}(s) \cup \{s\}} P_a^{\text{birth}}$$

and the routing-eligible evidence set  $\Omega_t(s) = \{x : q_t(s, x) \text{ exists and } x \notin \mathcal{B}_s\}$ . Routing credit for candidate  $s$  therefore draws exclusively from observations outside the batches that shaped its lineage.

The owner supplies the optimization observations accepted into the cache and, after selection, an independent frozen test.

### 2.2 RECURSIVE GENERATION-CERTIFICATION BOUNDARY

For a child proposed from parent  $s$  on batch  $P$ , define

$$X(s, P) = P \cup \bigcup_{a \in \text{Anc}(s) \cup \{s\}} P_a^{\text{birth}} \quad (1)$$

A legal admission batch satisfies  $A \cap X(s, P) = \emptyset$ . Proposal traces remain available for diagnosis. Certification for the child and its descendants uses batches disjoint from current and inherited birth provenance. In the running example, the three diagnostic cases explain the error, and a disjoint admission batch decides whether the new rule earns future search budget.

### 2.3 ROUTING LEGAL EVIDENCE

Let  $\mathcal{H}_t$  contain the lineage, birth provenance, committed cache, remaining budget, and persistent routing state. A routing action  $a_t = (\mathcal{A}_t, \mathbf{z}_t, Y_t)$  chooses active candidates, persistent parent slots, and a one-to-one proposal assignment from  $\mathcal{H}_t$ . The frozen owner test is reserved for terminal evaluation. Recursive exclusion supplies the generation–certification boundary. Instance-relative credit organizes uneven partial evidence as frontiers change. Persistent proposal slots and a coupled one-to-one assignment coordinate concurrent repair opportunities, and each fixed child receives an independently sampled admission batch.

A feasible router preserves the recursive boundary, the logical budget  $B$ , one-commit cache semantics, and terminal owner separation. Its optimal policy minimizes expected simple regret for the returned program.

## 3 COMPASS: ROUTING LEGAL EVIDENCE THROUGH SEARCH

One COMPASS wave carries evidence through a complete lifecycle. A proposal batch reveals a failure pattern and shapes a child. Equation (1) removes the evidence that shaped its lineage from the child’s legal admission pool. A legal admission batch then judges the fixed child, eligible outcomes enter the committed cache once, and the new state determines which lineages and tasks receive the next budget. The method therefore couples provenance, reuse, and allocation rather than treating them as separate heuristics.

### 3.1 FROM PROVENANCE TO COMMITTED EVIDENCE

For parent  $s$  and proposal batch  $P$ , COMPASS samples admission instances only outside  $X(s, P)$ . The admission identities, references, and outcomes remain hidden from the generator until the child text is fixed. The proposal rollout may update the existing parent, but  $P$  becomes the child’s birth provenance and is therefore excluded from that child’s routing-eligible set together with all inherited birth batches. At birth, the child enters the committed table only through its legal admission evaluation. Later owner-accepted executions may enlarge its routing evidence whenever their instances remain outside the same lineage birth set. This lets proposal traces retain their high-bandwidth diagnostic role while admission supplies a new observation that can certify the revision.

Let  $\mathcal{F}_t(x)$  be the owner frontier among candidates for which  $x \in \Omega_t(s)$ . Every current frontier owner counts as a frontier outcome, irrespective of the frontier’s raw score. Define the legally observed comparison set and its frontier count by

$$\mathcal{C}_t(x) = \{u : x \in \Omega_t(u)\}, \quad N_x = |\mathcal{C}_t(x)|, \quad y_{sx} = \mathbf{1}\{s \in \mathcal{F}_t(x)\}, \quad K_x = \sum_{u \in \mathcal{C}_t(x)} y_{ux}.$$

When  $N_x \geq 2$ , candidate  $s$  receives the leave-one-skill-out frontier residual

$$r_{sx} = y_{sx} - \frac{K_x - y_{sx}}{N_x - 1} = \frac{N_x y_{sx} - K_x}{N_x - 1}.$$

Only comparable exposures enter the aggregate:

$$\begin{aligned} \Omega_t^{\text{cmp}}(s) &= \{x \in \Omega_t(s) : N_x \geq 2\}, \quad E_s^{\text{cmp}} = |\Omega_t^{\text{cmp}}(s)|, \\ R_s &= \sum_{x \in \Omega_t^{\text{cmp}}(s)} r_{sx}, \quad G_s = \frac{R_s}{E_s^{\text{cmp}}}. \end{aligned} \tag{2}$$

Thus  $G_s$  records how often  $s$  reaches the current frontier relative to the other legally observed candidates on the same instances. An observation with  $N_x = 1$  remains in the committed table but contributes no relative credit until another candidate is legally observed on  $x$ . Missing entries remain

missing, and  $G_s$  is undefined when  $E_s^{\text{cmp}} = 0$  rather than being set to zero. Whenever a commit changes an observation or an owner frontier, COMPASS recomputes  $N_x$ ,  $K_x$ , all affected residuals, and the resulting  $G_s$  values.

### 3.2 A CREDIT-RANKED SEARCH FRONTIER

Let  $\mathcal{R}_t = \{s : E_s^{\text{cmp}} > 0\}$  be the candidates with defined credit. Parent–descendant relations do not remove candidates from this domain. Let  $\rho \in \mathbb{N}_+$  be the active-rank cutoff. If  $|\mathcal{R}_t| \geq \rho$ , let  $g_\rho$  be the  $\rho$ -th largest  $G_s$  and retain every candidate with  $G_s \geq g_\rho$ ; otherwise retain all of  $\mathcal{R}_t$ . The active set  $\mathcal{A}_t$  can therefore contain more than  $\rho$  programs at a boundary tie, and a parent and its descendant may remain active simultaneously. If no candidate has defined credit, COMPASS falls back to the seed.

The resulting active set is sorted by decreasing  $G_s$ , with earlier candidate index as the deterministic exact-tie order. Let  $w \in \mathbb{N}_+$  be the wave width. A persistent cyclic cursor draws  $w$  *slot identities* from this order. If  $|\mathcal{A}_t| < w$ , parent labels repeat but slot identities do not. The queue therefore preserves coverage across waves while allowing the current frontier to concentrate work.

### 3.3 TURNING CREDIT INTO PROPOSAL WORK AND LEGAL ADMISSION

Candidate credit decides which candidates own the wave’s persistent slot rights; it is not reused as a batch score. For candidate  $s$  and proposal minibatch  $P$ , define the recorded repair opportunity

$$H(s, P) = \frac{1}{|P|} \sum_{x \in P} \begin{cases} [\max_{r \in \mathcal{A}_t: q_t(r, x) \text{ finite}} q_t(r, x) - q_t(s, x)]_+, & q_t(s, x) \text{ finite,} \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

The zero branch is an opportunity weight, not an imputed failure. Let  $\mathcal{I}_w = \{0, \dots, w-1\}$  index the drawn slots  $z_i$  and proposal minibatches  $P_j$ . COMPASS allocates them by the maximum-weight binary program

$$\begin{aligned} Y^* \in \arg \max_{Y \in \{0,1\}^{w \times w}} \quad & \sum_{i,j \in \mathcal{I}_w} H(z_i, P_j) y_{ij} \\ \text{s.t.} \quad & \sum_{j \in \mathcal{I}_w} y_{ij} = 1 \quad \forall i, \quad \sum_{i \in \mathcal{I}_w} y_{ij} = 1 \quad \forall j. \end{aligned} \quad (4)$$

Thus distinct slots receive distinct proposal rights even when two slots carry the same parent label, and the wave realizes the largest recorded repair opportunity available to those slots and batches.

Proposal evaluation can update the existing parent and leaves a set of surviving proposal tasks. After these parent observations are committed, write  $m \leq w$  for the number of surviving tasks, with parent  $s_i$  and proposal batch  $P_i$ . For each task independently, the owner samples an admission batch  $A_i$  satisfying

$$A_i \cap X(s_i, P_i) = \emptyset.$$

The owner freezes  $A_i$  and, for each  $x \in A_i$ , selects a reference from the current frontier owners with defined  $G$ , preferring the largest  $G$  and using frozen randomness for an exact tie. If no such comparable owner exists, the sampled parent  $s_i$  is the reference. Denote this choice by  $r_i(x)$ . Missing reference observations are evaluated and maximum-bound on their existing programs before the child is generated, while admission identities, references, and outcomes remain hidden from the generator.

After the generator fixes child  $p_i$  from  $(s_i, P_i)$ , the owner evaluates the child and its references on the same batch and applies the paired aggregate gate

$$\Delta_i = \sum_{x \in A_i} [q(p_i, x) - q(r_i(x), x)], \quad \text{admit}(p_i) \iff \Delta_i > 0. \quad (5)$$

Maximum-bound proposal and admission-reference observations update existing programs, while only lineage-eligible entries affect their credit; a child’s own admission evidence enters the cache only with an owner-accepted child. COMPASS then recomputes every affected  $G_s$ , the tie-inclusive active envelope, and the queue. At the budget boundary, the maximal defined  $G_s$  is returned, with earliest index breaking an exact tie; if every  $G_s$  is undefined, the seed is returned. The terminal artifact alone enters independent owner evaluation. Expanded execution and fallback details appear in Section B.

## 4 HOW COMPASS TURNS EVIDENCE INTO SEARCH

To see how the components fit together, follow one revision through the search loop. A proposal batch reveals a parent’s failures and shapes a child. Since those examples helped create the child, different evidence must establish that the revision is useful. The resulting fresh outcomes are compared with other candidates on the same instances, converted into routing credit, and spent on the next set of proposal opportunities. The formal statements below support three intuitions needed to read this loop: admission tests a fixed revision, credit records relative distinctions rather than raw successes, and a fixed rollout budget must trade search breadth against decision precision.

### 4.1 A REVISION IS JUDGED ON FRESH PAIRED EVIDENCE

A revision can look perfect on the examples it was written to solve while changing nothing elsewhere. This makes a generation–certification boundary essential rather than cosmetic.

**Proposition 1** (Self-certification can be arbitrarily optimistic). *For every admission size  $n \geq 1$  and  $\epsilon > 0$ , there is a task distribution and a proposal rule that observes  $n$  examples, constructs a program, and scores one when those examples are reused for certification, although the program’s population utility is at most  $\epsilon$ .*

The construction in Section C memorizes the observed examples. COMPASS prevents that evidence from returning as certification for either the child that used it or later descendants that can inherit the resulting behavior.

**Fresh-pair principle.** The proposal batch explains how to change the parent; a disjoint admission batch decides whether that change helps. Before generation, the owner freezes the legal admission batch and its per-instance references and hides both from the generator. The gate in Equation (5) then compares the fixed child and its references on the same fresh instances. With  $n$  bounded pairs, the uncertainty of this paired advantage shrinks proportionally to  $1/\sqrt{n}$ . Thus additional admission evidence makes the decision less sensitive to a lucky minibatch. The exact non-anticipation statement and concentration bound appear in Theorem 1.

### 4.2 FRESH OUTCOMES BECOME INSTANCE-RELATIVE CREDIT

Admission decides whether a particular child enters the pool. Routing asks a different question: which accepted candidates have shown distinctions worth pursuing further? COMPASS answers it by comparing candidates on the same instance before aggregating across the partial cache.

**Proposition 2** (Instance-wise credit conservation). *For every instance  $x$  with  $N_x \geq 2$ , the residuals defined immediately before Equation (2) satisfy*

$$\sum_{s \in \mathcal{C}_t(x)} r_{sx} = 0.$$

This conservation law makes the score a contrast rather than a count. If all  $N_x$  compared candidates share the current frontier, every residual is zero: an instance that separates nobody raises nobody’s routing priority. If one candidate uniquely owns the frontier, it receives residual one and each of the other  $N_x - 1$  candidates receives  $-1/(N_x - 1)$ . Thus the same frontier event means more when it distinguishes a candidate from its observed peers.

These residuals are not immutable rewards. When a new committed observation changes  $\mathcal{F}_t(x)$ , COMPASS recomputes  $K_x$ , every affected  $r_{sx}$ , and the corresponding  $G_s$ . An instance with only one legal observation remains cached but begins contributing relative credit only after another candidate becomes comparable there.

Ancestry imposes no separate survival mask; it determines which observations a candidate may claim. Once  $G_s$  is defined, the tie-inclusive Top- $\rho$  rule ranks every candidate in one pool, so a parent and its descendant can remain active together. A persistent cursor then converts this credit order into proposal-slot rights without repeatedly resetting attention to the first candidate.

### 4.3 CREDIT DIRECTS THE NEXT WAVE

Three signals play different roles in the next wave. First,  $G_s$  determines which candidates receive the  $w$  persistent slots. Second, the recorded repair opportunity  $H(s, P)$  chooses which distinct proposal batch each slot receives through the exact binary assignment in Equation (4). Credit therefore chooses *whose* search continues, while  $H$  chooses *where* each proposal attempt is spent.

Admission is not a second allocation problem. Each surviving child receives its own independently sampled legal batch and frozen references, and the paired gate in Equation (5) decides whether it joins the pool. Accepted evidence can change the frontiers on its instances, which redistributes the corresponding residuals and updates the next credit order. The resulting feedback loop is

$$\begin{aligned} \text{fresh outcomes} &\longrightarrow G_s \longrightarrow \text{proposal-slot rights} \\ &\longrightarrow H\text{-matched proposals} \longrightarrow \text{fresh paired admission.} \end{aligned}$$

At the budget boundary, the maximal defined  $G_s$  is returned, with the seed used only when every credit remains undefined. The appendix records cyclic slot coverage and exact optimality of the proposal assignment.

### 4.4 WHY BREADTH AND CONFIDENCE COMPETE

Let  $\mathcal{P}_B$  be the candidates discovered within budget  $B$  and let  $\hat{p}_B$  be COMPASS’s actual  $G_s$ -selected terminal program. Its regret has the exact decomposition

$$R^* - R(\hat{p}_B) = \underbrace{R^* - \max_{p \in \mathcal{P}_B} R(p)}_{\text{search gap}} + \underbrace{\max_{p \in \mathcal{P}_B} R(p) - R(\hat{p}_B)}_{\text{selection gap}}. \quad (6)$$

The first term falls only when search discovers stronger candidates; the second falls only when the available evidence identifies a strong discovered candidate. A fixed rollout budget must support both.

The following calculation exposes the design law behind this breadth–confidence tradeoff.

**Proposition 3** (Fresh-evidence breadth–confidence tradeoff). *For any  $M, n \in \mathbb{N}_+$  and  $\delta \in (0, 1)$ , let  $p_1, \dots, p_M$  be adaptively generated candidates. After  $p_i$  is fixed, suppose it receives  $n$  conditionally iid owner rewards in  $[0, 1]$  from one common owner law with mean  $R(p_i)$ . Let  $\hat{R}_n(p_i)$  be their mean and define*

$$\tilde{p}_{M,n} \in \arg \max_{i \leq M} \hat{R}_n(p_i).$$

Then, with probability at least  $1 - \delta$ ,

$$R^* - R(\tilde{p}_{M,n}) \leq \underbrace{R^* - \max_{i \leq M} R(p_i)}_{\text{search gap}} + \underbrace{2\sqrt{\frac{\log(2M/\delta)}{2n}}}_{\text{selection noise}}. \quad (7)$$

The bound yields a direct design principle: more distinct proposal attempts can reduce the search gap, while more fresh support for each candidate shrinks the uncertainty of choosing among them. COMPASS addresses both demands explicitly: credit-ranked slots and exact proposal matching spend budget on search breadth, while provenance-legal paired admission spends evidence on each decision. The resulting  $G_s$  order directs both continued search and terminal selection.

## 5 EXPERIMENTS

### 5.1 EVALUATION QUESTIONS

We evaluate whether provenance-constrained routing improves held-out owner utility under a fixed budget and whether the result persists across models and tasks.

## 5.2 TASKS, MODELS, AND OWNER EVALUATION

Our study spans AIME-2025 and LiveBench-Math mathematical reasoning, IFBench instruction following with unseen constraint types, HotPotQA question answering, HoVer multi-hop fact verification, PUPA privacy-conscious delegation (Pyatkin et al., 2025; Balunović et al., 2025; White et al., 2025; Yang et al., 2018; Jiang et al., 2020; Li et al., 2025). We evaluate Qwen3-8B (Yang et al., 2025) and GPT-5.4-nano. PUPA retains its benchmark-owned GPT-4.1-mini judge for response quality and PII leakage; this fixed scorer is not an optimized task or reflection model.

Official evaluation is separated from optimization. The Qwen3-8B AIME variant is evaluated on all 150 owner problems. HoVer uses 150/300/300 train, validation, and test instances with a 7,051-call budget; LiveBench-Math uses its frozen 126-problem test with a 1,839-call budget. No official test result feeds optimization.

## 5.3 QWEN3-8B: COMPARISON ACROSS SIX TASKS

Table 1 compares completed Qwen3-8B owner evaluations on all six shared tasks. The comparison rows reproduce the published GEPA study; the COMPASS row uses the frozen task-specific endpoint for each task. COMPASS has the highest six-task aggregate and the highest score on five tasks. GEPA remains higher on PUPA.

Table 1: **Qwen3-8B owner evaluation across all shared tasks (%)**. Each cell reports the official task aggregate. Aggregate is the unweighted mean over the six tasks, and  $\Delta$  Seed is the aggregate improvement in percentage points. Comparison rows are published GEPA-study results; TextGrad and Trace were not reported for Qwen3-8B.

| Method     | HotPotQA     | IFBench      | HoVer        | PUPA         | AIME-2025    | LiveBench-Math | Aggregate    | $\Delta$ Seed |
|------------|--------------|--------------|--------------|--------------|--------------|----------------|--------------|---------------|
| Seed       | 42.33        | 36.90        | 35.33        | 80.82        | 27.33        | 48.70          | 45.23        | –             |
| GRPO       | 43.33        | 35.88        | 38.67        | 86.66        | 38.00        | 51.26          | 48.91        | +3.68         |
| MIPROv2    | 55.33        | 36.22        | 47.33        | 81.55        | 20.00        | 46.60          | 47.84        | +2.61         |
| GEPA       | 62.33        | 38.61        | 52.33        | <b>91.85</b> | 32.00        | 51.95          | 54.85        | +9.62         |
| GEPA+Merge | 64.33        | 28.23        | 51.67        | 86.26        | 32.00        | 51.95          | 52.40        | +7.17         |
| COMPASS    | <b>64.67</b> | <b>43.37</b> | <b>54.00</b> | 86.91        | <b>57.33</b> | <b>66.67</b>   | <b>62.16</b> | <b>+16.93</b> |

The Qwen3-8B COMPASS endpoints select candidates 55, 78, 76, 34, 15, and 30 for HotPotQA, IFBench, HoVer, PUPA, AIME-2025, and LiveBench-Math, respectively. Their owner evaluations cover 300, 294, 300, 221, 150, and 126 test instances. In every case, the official test split is frozen before evaluation and does not participate in candidate selection.

## 5.4 GPT-5.4-NANO: SAME-MODEL COMPARISON

Table 2 reports the completed GPT-5.4-nano matrix. Every optimized task and reflection model uses GPT-5.4-nano; PUPA keeps the fixed benchmark scorer described above. COMPASS has the highest six-task aggregate and the highest score on four tasks. MIPROv2 is higher on HotPotQA, and GEPA is higher on HoVer.

Table 2: **GPT-5.4-nano owner evaluation across all shared tasks (%)**. Each cell reports the official task aggregate. Aggregate is the unweighted mean over the six tasks, and  $\Delta$  Seed is the aggregate improvement in percentage points. GEPA’s AIME-2025 cell uses its first completed owner evaluation. Methods without a completed same-model result are omitted.

| Method  | HotPotQA     | IFBench      | HoVer        | PUPA         | AIME-2025    | LiveBench-Math | Aggregate    | $\Delta$ Seed |
|---------|--------------|--------------|--------------|--------------|--------------|----------------|--------------|---------------|
| Seed    | 31.33        | 48.98        | 45.33        | 74.86        | 15.33        | 35.09          | 41.82        | –             |
| MIPROv2 | <b>61.67</b> | 47.79        | 53.00        | 75.45        | 30.00        | 43.66          | 51.93        | +10.11        |
| GEPA    | 57.33        | 48.30        | <b>61.33</b> | 81.09        | 34.00        | 35.03          | 52.85        | +11.03        |
| COMPASS | 60.00        | <b>51.36</b> | 60.33        | <b>84.14</b> | <b>42.00</b> | <b>45.03</b>   | <b>57.14</b> | <b>+15.32</b> |

## 6 RELATED WORK

**Reflective textual-program optimization.** Reflexion and Self-Refine use language feedback to revise behavior, while TextGrad, Trace/OptoPrime, MIPROv2, and GEPA optimize textual components from execution evidence (Shinn et al., 2023; Madaan et al., 2023; Yuksekogonul et al., 2025; Cheng et al., 2024; Opsahl-Ong et al., 2024; Agrawal et al., 2025). COMPASS focuses on evidence routing: which observations may certify a lineage, how a candidate’s frontier contribution should be measured against other legally observed candidates on the same instance, and where the next proposal budget should flow.

**Evolution, illumination, and skill search.** Evolutionary prompt and skill methods maintain multiple candidates to avoid a single greedy trajectory (Agrawal et al., 2025; Mi et al., 2026; Moll et al., 2026; Zhang et al., 2026; Wang et al., 2026b). COMPASS differs by keeping parents and descendants in one credit-ranked candidate pool and using a tie-inclusive Top- $\rho$  envelope rather than an ancestry-based handoff.

**Adaptive evaluation and budget allocation.** TRIPLE, POES, and Bayesian-Agent allocate evaluation or retrieval effort under limited budgets (Shi et al., 2024; Ma et al., 2026; Wu et al., 2026); TextBO and APEX optimize evaluation efficiency and development-example choice (Kang & Yoganarasimhan, 2026; Wang et al., 2026a). COMPASS couples instance-relative candidate credit, exact one-to-one proposal work, and independently sampled legality-first admission in one committed state. Its credit is a within-instance frontier residual rather than a raw aggregate reward or frontier frequency.

**Generalization after adaptive search.** Repeated adaptive use of optimization data can widen the gap between selected validation performance and fresh test performance (Dwork et al., 2015). GEPA addresses this empirically through reusable rules and branch diversity (Agrawal et al., 2025). COMPASS adds recursive certification separation and an instance-relative frontier residual that is recomputed as the committed comparison state changes. Fixed owner tests measure the resulting endpoint.

## 7 CONCLUSION

We have treated textual-program evolution as a fixed-budget information problem. Fresh support reduces decision noise, independent candidates increase search bandwidth, and provenance determines whether reused evidence may certify a lineage. COMPASS operationalizes this tradeoff through recursive exclusion, dynamically recomputed instance-relative credit, a tie-inclusive candidate envelope, exact proposal assignment, and direct legal admission. Completed Qwen3-8B owner evaluations span IFBench, AIME-2025, HoVer, and LiveBench-Math; the GPT-5.4-nano matrix spans all six shared tasks. COMPASS contributes the routing state rather than a new text generator. This state determines which evidence creates credit and which proposal work that credit may purchase.

## REFERENCES

- Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning, 2025. URL <https://arxiv.org/abs/2507.19457>. ICLR 2026 Oral.
- Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Aime-2025, February 2025. URL [https://huggingface.co/datasets/MathArena/aime\\_2025](https://huggingface.co/datasets/MathArena/aime_2025). Dataset from MathArena: Evaluating LLMs on Uncontaminated Math Competitions.
- Ching-An Cheng, Allen Nie, and Adith Swaminathan. Trace is the next autodiff: Generative optimization with rich feedback, execution traces, and llms, 2024. URL <https://arxiv.org/abs/2406.16218>.

- Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. Generalization in adaptive data analysis and holdout reuse. In *Advances in Neural Information Processing Systems*, volume 28, 2015. URL <https://proceedings.neurips.cc/paper/2015/hash/bad5f33780c42f2588878a9d07405083-Abstract.html>.
- Yichen Jiang, Shikha Bordia, Zheng Zhong, Charles Dognin, Maneesh Singh, and Mohit Bansal. HoVer: A dataset for many-hop fact extraction and claim verification. In Trevor Cohn, Yulan He, and Yang Liu (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 3441–3460, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.findings-emnlp.309. URL <https://aclanthology.org/2020.findings-emnlp.309/>.
- Enoch Hyunwook Kang and Hema Yoganarasimhan. TextBO: Bayesian optimization in language space for eval-efficient self-improving AI. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2511.12063>.
- Siyao Li, Vethavikashini Chithra Raghuram, Omar Khattab, Julia Hirschberg, and Zhou Yu. PA-PILLON: Privacy preservation from Internet-based and local language model ensembles. In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.), *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 3371–3390, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.173. URL <https://aclanthology.org/2025.naacl-long.173/>.
- Xiaoyu Ma, Yiwen Li, Haoyue Liu, Zhichao Wang, Ye Chen, Yongxin Guo, and Xiaoying Tang. Select smarter, not more: Prompt-aware evaluation scheduling with submodular guarantees, 2026. URL <https://arxiv.org/abs/2604.11328>.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegraffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 46534–46594. Curran Associates, Inc., 2023. URL [https://proceedings.neurips.cc/paper\\_files/paper/2023/file/91eddf07232fb1b55a505a9e9f6c0ff3-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2023/file/91eddf07232fb1b55a505a9e9f6c0ff3-Paper-Conference.pdf).
- Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Skill-Pro: Learning reusable skills from experience via non-parametric PPO for LLM agents, 2026. URL <https://arxiv.org/abs/2602.01869>. ICML 2026 Spotlight.
- Johannes Moll, Jean-Philippe Corbeil, Jiazhen Pan, Martin Hadamitzky, Daniel Rueckert, Lisa Adams, and Keno Bressen. GRASP: Gated regression-aware skill proposer for self-improving LLM agents, 2026. URL <https://arxiv.org/abs/2605.29668>.
- Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 9340–9366, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.525. URL <https://aclanthology.org/2024.emnlp-main.525/>.
- Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hannaneh Hajishirzi. Generalizing verifiable instruction following, 2025. URL <https://arxiv.org/abs/2507.02833>.
- Chengshuai Shi, Kun Yang, Zihan Chen, Jundong Li, Jing Yang, and Cong Shen. Efficient prompt optimization through the lens of best arm identification. In *Advances in Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=FLNnlFBGMo>.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In *Proceedings of the 37th International*

---

540 *Conference on Neural Information Processing Systems, NIPS '23*, Red Hook, NY, USA, 2023.  
541 Curran Associates Inc.

542

543 Fei Wang, Si Si, Cho-Jui Hsieh, and Inderjit S. Dhillon. APEX: Automated prompt engineering  
544 expert with dynamic data selection, 2026a. URL <https://arxiv.org/abs/2606.11459>.

545 Zefeng Wang, Minxi Yan, Jinhe Bi, Sikuan Yan, Volker Tresp, and Yunpu Ma. MetaSkill-Evolve:  
546 Recursive self-improvement of LLM agents via two-timescale meta-skill evolution, 2026b. URL  
547 <https://arxiv.org/abs/2607.05297>.

548

549 Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-  
550 Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, Shubh-Agrawal, Sandeep Singh Sandha, Siddhartha  
551 Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum.  
552 Livebench: A challenging, contamination-limited llm benchmark, 2025. URL <https://arxiv.org/abs/2406.19314>.

553

554 Xiaojun Wu, Cehao Yang, Honghao Liu, Xueyuan Lin, Wenjie Zhang, Zhichao Shi, Xuhui Jiang,  
555 Chengjin Xu, Jia Li, and Jian Guo. Bayesian-Agent: Posterior-guided skill evolution for LLM  
556 agent harnesses, 2026. URL <https://arxiv.org/abs/2606.08348>.

557

558 An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang  
559 Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu,  
560 Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin  
561 Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang,  
562 Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui  
563 Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang  
564 Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger  
565 Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan  
566 Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.

567

568 Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov,  
569 and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question  
570 answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

571

572 Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin,  
573 and James Zou. Optimizing generative ai by backpropagating language model feedback. *Nature*,  
574 639:609–616, 2025.

575

576 Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou,  
577 Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S.  
578 Yu. CoEvoSkills: Self-evolving agent skills via co-evolutionary verification, 2026. URL  
579 <https://arxiv.org/abs/2604.01687>.

580

581

582

583

584

585

586

587

588

589

590

591

592

593

## A NOTATION AND EXACT STATE

### A.1 ELIGIBLE EXPOSURE AND FRONTIERS

At a fixed routing state  $t$ , reuse the lineage-eligible set  $\Omega_t(s) = \{x : q_t(s, x) \text{ exists and } x \notin \mathcal{B}_s\}$  from Section 2. The owner frontier is

$$\mathcal{F}_t(x) = \arg \max_{s: x \in \Omega_t(s)} q_t(s, x),$$

with all exact ties retained and every frontier owner counted, regardless of the frontier’s raw score. For each instance, define

$$\mathcal{C}_t(x) = \{u : x \in \Omega_t(u)\}, \quad N_x = |\mathcal{C}_t(x)|, \quad y_{sx} = \mathbf{1}\{s \in \mathcal{F}_t(x)\}, \quad K_x = \sum_{u \in \mathcal{C}_t(x)} y_{ux}.$$

When  $N_x \geq 2$ , the per-observation credit is

$$r_{sx} = y_{sx} - \frac{K_x - y_{sx}}{N_x - 1}.$$

The candidate aggregate is

$$\Omega_t^{\text{cmp}}(s) = \{x \in \Omega_t(s) : N_x \geq 2\}, \quad E_s^{\text{cmp}} = |\Omega_t^{\text{cmp}}(s)|, \quad R_s = \sum_{x \in \Omega_t^{\text{cmp}}(s)} r_{sx}, \quad G_s = R_s / E_s^{\text{cmp}}.$$

An  $N_x = 1$  observation remains committed but does not enter the aggregate. Whenever a new finite observation changes  $\mathcal{C}_t(x)$  or  $\mathcal{F}_t(x)$ , the router recomputes  $N_x$ ,  $K_x$ , and every affected candidate contribution on that instance. Candidates with  $E_s^{\text{cmp}} = 0$  have undefined credit; if every credit is undefined, routing and final selection fall back to the seed.

### A.2 RECURSIVE PROPOSAL-LINEAGE EXCLUSION

If parent  $s$  is proposed on batch  $P$ , the prospective child cannot use its current proposal instances or any ancestor birth-proposal instances as admission evidence. Let  $P_a^{\text{birth}}$  be the recorded proposal batch that created accepted candidate  $a$ , with  $P_0^{\text{birth}} = \emptyset$  for the seed. The excluded set is

$$X(s, P) = P \cup \bigcup_{a \in \text{Anc}(s) \cup \{s\}} P_a^{\text{birth}}.$$

Admission batches are not themselves added to this birth provenance. This keeps proposal feedback recursively separate from the evidence that decides whether the next child is admitted. The lineage birth set  $\mathcal{B}_s$  governs which committed observations may contribute to candidate  $s$ ’s routing key;  $X(s, P)$  extends that exclusion to the prospective child’s admission.

### A.3 CHECKPOINT STATE

The optimizer checkpoint records the accepted programs, parent relation, committed reward cache, frontier map, logical call count, active-queue cursor, proposal assignments, sampled admission batches, and seeded random states. The queue cursor is interpreted modulo the current active order after every recomputation.

## B ALGORITHMIC DETAILS

### B.1 PROPOSAL ALLOCATION IMPLEMENTATION

Equations (3) and (4) define the scientific proposal objective. The implementation builds the complete slot–batch weight matrix from the committed cache and solves the resulting maximum-weight assignment exactly. Duplicate parent labels remain distinct left-side slot vertices, and the operation never fills missing cache entries.

### B.2 DIRECT LEGAL ADMISSION IMPLEMENTATION

For each of the  $m \leq w$  surviving proposal tasks, the owner directly samples one batch  $A_i$  from that task’s admission sampler after removing every instance in  $X(s_i, P_i)$ . For each instance in the sampled batch, the owner selects the defined- $G$  frontier owner with largest  $G$  as reference, using frozen randomness for an exact tie and the sampled parent when no comparable owner exists. The resulting plan is hidden from the generator, required missing reference observations are filled on the existing reference programs, and the fixed child is admitted exactly when the paired aggregate in Equation (5) is positive.

### B.3 WAVE EXECUTION

---

#### Algorithm 1 One COMPASS wave

---

**Require:** committed routing state  $\mathcal{H}_t$ , wave width  $w$ , active cutoff  $\rho$   
**Ensure:** updated committed state  $\mathcal{H}_{t+1}$

- 1: Recompute every defined  $G_s$  from the committed cache
- 2:  $\mathcal{A}_t \leftarrow$  tie-inclusive Top- $\rho$  candidates, sorted by  $G_s$  and candidate index
- 3:  $z_{1:w} \leftarrow \text{CYCLICSLOTS}(\mathcal{A}_t, w)$
- 4: Draw proposal minibatches  $P_{1:w}$  and form  $M_{ij} \leftarrow H(z_i, P_j)$
- 5:  $Y^* \leftarrow \text{MAXWEIGHTASSIGNMENT}(M)$
- 6: **for all** assigned parent-proposal pairs  $(s_i, P_i)$  **in parallel do**
- 7:     Evaluate  $s_i$  on  $P_i$  and construct its reflection data
- 8:     Maximum-bound and commit the resulting parent observations
- 9: **end for**
- 10:  $\mathcal{T} \leftarrow$  surviving proposal tasks;  $m \leftarrow |\mathcal{T}| \leq w$
- 11: **for all**  $(s_i, P_i) \in \mathcal{T}$  **in parallel do**
- 12:      $A_i \leftarrow \text{SAMPLEADMISSION}(s_i, P_i)$  subject to  $A_i \cap X(s_i, P_i) = \emptyset$
- 13:      $r_i \leftarrow \text{BINDREFERENCES}(A_i)$ ; freeze and hide  $(A_i, r_i)$  from the generator
- 14:     Evaluate and maximum-bound any missing reference observations
- 15:      $p_i \leftarrow \text{REFLECT}(s_i, P_i)$ ; evaluate  $p_i$  on  $A_i$
- 16:      $\Delta_i \leftarrow \sum_{x \in A_i} [q(p_i, x) - q(r_i(x), x)]$
- 17:     **if**  $\Delta_i > 0$  **then**
- 18:         Commit  $p_i$  and its lineage-eligible admission evidence
- 19:     **end if**
- 20: **end for**
- 21: Update the logical budget, frontiers, credits, and queue cursor; persist  $\mathcal{H}_{t+1}$

---

Repeat Algorithm 1 until the owner budget boundary. Because the logical budget is checked between waves, the final wave may overshoot; we report both target and actual calls. Request-level retries add no logical task result.

## C PROOFS FOR EVIDENCE, CREDIT, AND REGRET

The results below formalize three parts of the mechanism: recursive separation creates a conditionally fresh paired admission channel, relative credit is conserved within each comparable instance, and a canonical reference selector exposes the fixed-budget pressure between search breadth and decision precision.

### C.1 WHY GENERATION AND CERTIFICATION MUST SEPARATE

*Proof of Proposition 1.* Let the task distribution be uniform on a domain of size  $K \geq n/\epsilon$ . After observing  $S = (x_1, \dots, x_n)$ , let the proposal rule construct a program whose reward is one exactly on the distinct elements appearing in  $S$  and zero elsewhere. Every reused certification example belongs to that set, so the empirical reward on  $S$  is one. The population reward is the fraction of the domain covered by the observed set, at most  $n/K \leq \epsilon$ .  $\square$

The construction captures the mechanism in its simplest form: certification is informative only when it contributes evidence beyond what already shaped the candidate. Recursive birth provenance extends this separation through the lineage, because a descendant can retain behavior adapted to an ancestor's proposal batch.

### C.2 RECURSIVE EXCLUSION CREATES A PAIRED CERTIFICATION CHANNEL

**Theorem 1** (Fresh paired admission). *Let  $\mathcal{V}$  be the information visible to the generator when child  $p$  is fixed from parent  $s$  and proposal batch  $P$ , and suppose that  $p$  is  $\mathcal{V}$ -measurable. Before generation, the owner binds an admission batch  $A = (X_1, \dots, X_n)$  outside  $X(s, P)$  and binds each reference  $r(X_j)$  from the committed pre-child state and frozen tie-breaking randomness. Admission identities, references, and outcomes are not visible in  $\mathcal{V}$ . Then*

$$A \cap P = \emptyset, \quad A \cap P_a^{\text{birth}} = \emptyset \quad \text{for every } a \in \text{Anc}(s) \cup \{s\}.$$

*After  $p$  is fixed, reveal the paired outcomes sequentially. Let  $\mathcal{H}_{j-1}$  contain the fixed child, the owner binding state, and the first  $j-1$  revealed pairs, and define*

$$Z_j = q(p, X_j) - q(r(X_j), X_j), \quad \mu_j = \mathbb{E}[Z_j \mid \mathcal{H}_{j-1}],$$

$$\hat{\mu}_A(p; r) = \frac{1}{n} \sum_{j=1}^n Z_j, \quad \bar{\mu}_A(p; r) = \frac{1}{n} \sum_{j=1}^n \mu_j.$$

Because  $Z_j \in [-1, 1]$ , for every  $\delta \in (0, 1)$ , with probability at least  $1 - \delta$ ,

$$|\hat{\mu}_A(p; r) - \bar{\mu}_A(p; r)| \leq \sqrt{\frac{2 \log(2/\delta)}{n}}. \quad (8)$$

For the  $i$ th task, the implemented gate is equivalently admit( $p_i$ )  $\iff \hat{\mu}_{A_i}(p_i; r_i) > 0$ .

*Proof of Theorem 1.* Because  $X(s, P)$  contains both  $P$  and every  $P_a^{\text{birth}}$  for  $a \in \text{Anc}(s) \cup \{s\}$ , legality  $A \cap X(s, P) = \emptyset$  gives the displayed disjointness relations. After the child is fixed, each paired difference  $Z_j$  lies in an interval of length two. Conditional Hoeffding–Azuma applied to  $Z_j - \mathbb{E}[Z_j \mid \mathcal{H}_{j-1}]$  gives

$$\Pr(|\hat{\mu}_A(p; r) - \bar{\mu}_A(p; r)| \geq \epsilon) \leq 2 \exp\left(-\frac{n\epsilon^2}{2}\right).$$

Setting the right-hand side to  $\delta$  proves Equation (8).  $\square$

Equation (1) supplies this separation through the entire birth lineage. COMPASS binds admission before child generation and hides admission identities and references from the generator. The resulting paired contrast is therefore a conditionally fresh information channel for the implemented owner gate.

### C.3 RELATIVE CREDIT IS CONSERVED INSTANCE-WISE

*Proof of Proposition 2.* For a fixed comparable instance  $x$ , use  $\sum_{s \in \mathcal{C}_t(x)} y_{sx} = K_x$  to obtain

$$\begin{aligned} \sum_{s \in \mathcal{C}_t(x)} r_{sx} &= K_x - \frac{\sum_{s \in \mathcal{C}_t(x)} (K_x - y_{sx})}{N_x - 1} \\ &= K_x - \frac{N_x K_x - K_x}{N_x - 1} = 0. \end{aligned}$$

The statement is per instance; the candidate averages  $G_s$  need not sum to zero because their comparable exposure sets and denominators can differ.  $\square$

### C.4 THE BANDWIDTH–VARIANCE REGRET ENVELOPE

To isolate the breadth–precision mechanism, consider a canonical fresh-certification reference model. Let  $c_{\text{prop}}$  denote the rollout cost of producing one candidate before certification, and let  $n \geq 1$  be its fresh admission support. A budget  $B$  can support

$$M_B(n) = \left\lfloor \frac{B}{c_{\text{prop}} + n} \right\rfloor$$

distinct proposal-and-certification attempts.

For Proposition 3, let  $\mathcal{F}_{i-1}$  contain the complete adaptive search history before candidate  $p_i$  is certified and let  $p_i$  be measurable with respect to that history. Conditional on  $\mathcal{F}_{i-1}$ , its  $n$  certification rewards are iid from the same owner law and are hidden until  $p_i$  is fixed. The fresh-mean reference selector is  $\tilde{p}_{M,n} \in \arg \max_i \hat{R}_n(p_i)$ .

*Proof of Proposition 3.* Conditional on  $\mathcal{F}_{i-1}$  and  $p_i$ , Hoeffding’s inequality bounds candidate  $i$ ’s empirical error by  $\epsilon$  with failure probability  $2e^{-2n\epsilon^2}$ . Taking expectations over the history preserves this bound, and a union bound over  $M$  candidates yields simultaneous error at most  $\epsilon = \sqrt{\log(2M/\delta)/(2n)}$ . Let  $p_M^*$  maximize population utility among the generated candidates. On this event,  $R(p_M^*) - R(\tilde{p}_{M,n}) \leq 2\epsilon$ . Adding  $R^* - R(p_M^*)$  gives Equation (7).  $\square$

Because rewards lie in  $[0, 1]$ , taking expectations and bounding the complement of the concentration event by one also gives

$$\mathbb{E}[R^* - R(\tilde{p}_{M,n})] \leq \mathbb{E}\left[R^* - \max_{i \leq M} R(p_i)\right] + 2\sqrt{\frac{\log(2M/\delta)}{2n}} + \delta.$$

The two terms isolate the reference tradeoff. Increasing  $n$  tightens each certification decision but lowers  $M_B(n)$ ; increasing bandwidth raises the chance of discovering a strong program but leaves less fresh support per candidate. Along a growing candidate sequence, the expected search-regret term is nonincreasing in  $M$ , whereas the displayed selection term decreases with  $n$ . In COMPASS, recursive exclusion, instance-relative candidate ranking, cyclic slots, and  $H$ -based proposal matching route the corresponding budget across fresh decisions and new proposal attempts.

---

## C.5 STRUCTURAL PROPERTIES OF THE ROUTER

**Proposition 4** (Cyclic slot coverage). *For a fixed ordered active set of size  $m$ , every block of  $m$  consecutive queue draws contains each active candidate exactly once, including when a width- $w$  wave wraps around the queue.*

*Proof of cyclic slot coverage.* Index the fixed active order by  $0, \dots, m - 1$  and let the persistent cursor be  $c$ . Consecutive draws are  $(c + r) \bmod m$ . For  $r = 0, \dots, m - 1$ , these residues are pairwise distinct and cover every index. Grouping the same sequence into blocks of width  $w$  does not alter the argument, including a block that wraps around the end of the queue.  $\square$

**Proposition 5** (Wave-optimal proposal matching). *Conditional on the  $w$  drawn parent slots and  $w$  proposal minibatches, Equation (4) maximizes total recorded repairable gap over all one-to-one slot-minibatch assignments.*

*Proof of wave-optimal proposal matching.* The  $w$  parent slots and  $w$  minibatches define a complete bipartite graph with edge weight  $H(z_i, P_j)$ . A feasible binary matrix in Equation (4) is exactly a perfect matching, or equivalently a permutation matrix. Selecting the maximum-weight feasible matrix therefore returns the maximum-weight matching. Repeated parent labels do not change the graph because their slot vertices remain distinct.  $\square$

## D DETAILED EXPERIMENTAL PROTOCOL

### D.1 METHOD IDENTITY

The method in Section 3 uses `leave_one_out_frontier_credit`, applies the tie-inclusive Top- $\rho$  cutoff without ancestor masking, preserves the exact proposal assignment in Equation (4), and samples admission batches directly per surviving proposal. If the final selector returns seed index zero, the protocol records a no-change endpoint and skips official evaluation; otherwise, it evaluates the selected artifact once.

### D.2 OWNER EVALUATION INTEGRITY

An endpoint enters a result table only when the optimization manifest is terminal, the exact selected candidate and its source/config/model identities are fixed, and the owner evaluation is complete. IFBench requires 294 finite example scores whose mean equals the aggregate. AIME requires all 150 owner problems. HoVer requires 300 finite binary scores, and LiveBench-Math requires all 126 frozen test positions under its owner aggregation.

Runs with an incorrect selected candidate, incompatible source identity, or provider exhaustion converted to score zero are not eligible evidence. Preflight results, intermediate checkpoints, and test-best oracle candidates are never substituted for the selected owner artifact.

### D.3 RESOURCE ACCOUNTING

We report logical task calls and final-wave overshoot separately from physical language-model calls. The latter include input/output tokens and route-priced cost. A local task deadline can produce an aligned failure observation; other provider, schema, serialization, and engineering errors terminate the owner batch. A resumable run must preserve the exact source, config, cache, checkpoint, and RNG identity.

## E BROADER IMPACT

Evidence routing can reduce the number of expensive environment executions needed to discover a useful textual program and can make search decisions easier to audit. The same efficiency can also optimize undesirable procedures or exploit imperfect rewards. Strict admission and held-out tests do not replace secure tool policies, sandboxing, red-team evaluation, or human review.

**Use of large language models.** Language models assisted with literature review, editing, code, and figure design. The authors verified every citation, algorithm, and artifact; model output was neither experimental evidence nor used to choose test candidates.